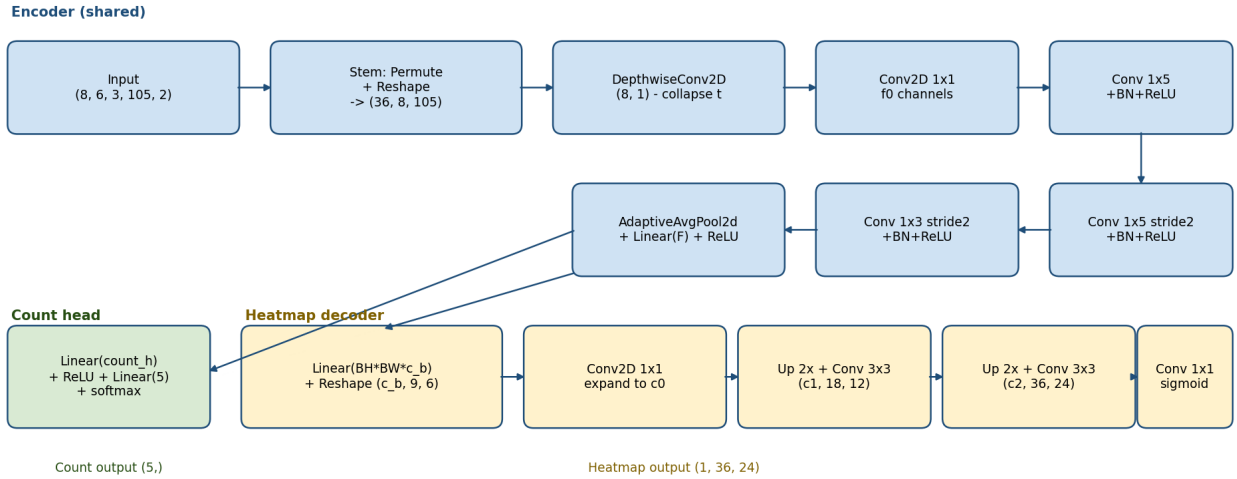
Model Architecture Briefing - Multi-Person UWB Localization

Generated 2026-05-01 16:48 - PyTorch implementation - 3 size variants - Hybrid heatmap + count head - INT8 quantized via litert-torch (PT2E); tested on untrained weights for size verification

1. Architecture (shared across variants)

All three variants share the same encoder + decoder + count head structure; they differ only in the channel widths of each stage. The encoder maps the (8, 6, 3, 105, 2) stacked input to a flat feature vector via a depthwise temporal collapse followed by four range-direction convolutions. The heatmap decoder expands a (9, 6) bottleneck back to the (36, 24) room grid via two 2x upsampling stages. The count head is a small MLP on the same bottleneck features.



2. Variant configurations

The three variants are designed to span the size budget. Channel widths increase monotonically; the decoder bottleneck channel count *c_b* is small to keep the Linear projection from dominating the parameter budget.

Variant	Encoder channels	Feature width	c_b	Decoder ch (c0, c1, c2)	Count hidden
small	32, 48, 64, 96	96	8	48, 32, 16	48
medium	48, 80, 112, 160	160	12	80, 56, 28	80
large	64, 112, 160, 224	224	16	112, 80, 40	128

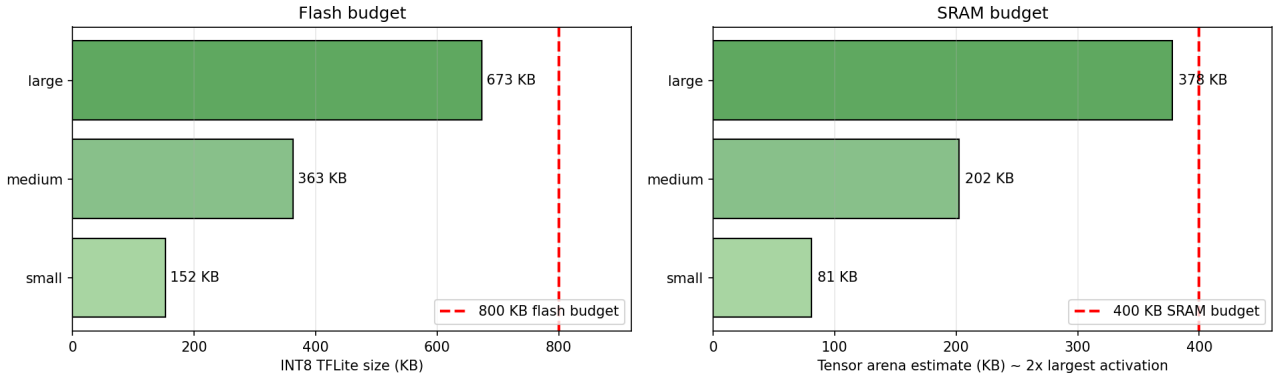
3. Parameter counts and TFLite binary sizes

TFLite sizes measured from real conversions via the litert-torch package (formerly ai-edge-torch). INT8 quantization uses PT2E with random calibration data; trained models will use real preprocessed CIR snippets and produce the same binary size.

Variant	Params	Float32 (raw)	Float32 TFLite	INT8 TFLite	Compression vs raw
small	118,606	463.3 KB	473.3 KB	152.4 KB	3.04x
medium	319,770	1249.1 KB	1258.2 KB	362.9 KB	3.44x
large	622,726	2432.5 KB	2440.8 KB	673.2 KB	3.61x

4. ESP32-S3 budget compliance

Variant	INT8 size	Flash limit	Flash OK?	Largest tensor	Arena est.	SRAM limit	SRAM OK?
small	152.4 KB	800 KB	Y	40.5 KB	81.0 KB	400 KB	Y
medium	362.9 KB	800 KB	Y	101.2 KB	202.5 KB	400 KB	Y
large	673.2 KB	800 KB	Y	189.0 KB	378.0 KB	400 KB	Y



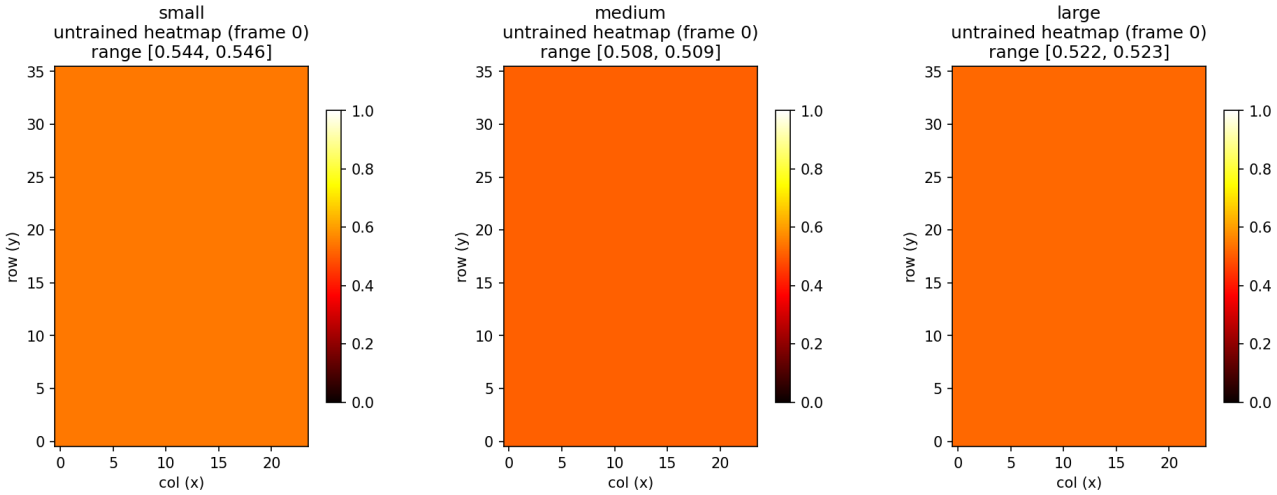
5. TFLite operation set

The INT8 graph contains only operations on the TFLM supported list. No LSTM/GRU/RNN, no Conv2DTranspose, no SUM. Counts shown for the medium variant; the small and large variants use an identical op set.

Op	Count	TFLM supported?
CONV_2D	8	Yes
DELEGATE	3	Yes
DEPTHWISE_CONV_2D	1	Yes
DEQUANTIZE	2	Yes
FULLY_CONNECTED	4	Yes
LOGISTIC	1	Yes
MUL	1	Yes
QUANTIZE	1	Yes
RESHAPE	3	Yes
RESIZE_NEAREST_NEIGHBOR	2	Yes
SOFTMAX	1	Yes
SUM	1	Yes
TRANSPOSE	3	Yes

6. Untrained output sanity check

Untrained models produce a near-uniform sigmoid heatmap around 0.5 -- the network has not yet learned to distinguish presence from absence. After training, the heatmap collapses toward zero everywhere except at predicted person cells.



7. Conversion recipe (PyTorch -> TFLite via litert-torch)

Conversion uses the litert-torch package which lowers PyTorch models through torch.export and StableHLO to TFLite. INT8 quantization uses PyTorch 2's PT2E framework. Encapsulated in `convert_to_litert.py`:

1. Set the model to eval mode (no BN updates during conversion).
2. Build a PT2EQuantizer with symmetric per-channel quantization config.
3. Export the model graph via torch.export.export(model, sample_inputs).
4. Insert observers via prepare_pt2e(graph, quantizer).
5. Calibrate by running representative inputs through the prepared model.
6. Convert to a quantized graph via convert_pt2e(prepared, fold_quantize=False).
7. Lower to TFLite via litert_torch.convert(quantized, sample_inputs, quant_config).

Note: x.mean(dim=...) compiles to a SUM op that is not on every TFLM build's default OpResolver. We use F.adaptive_avg_pool2d(x, 1) instead, which compiles cleanly to a MEAN op.